

Why your computer stopped getting faster

The single fact this whole course is built on.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

1971 to 2005: wait, and your code got faster

Clock speed doubled every two years. You did nothing.

Since 2005: nothing.
Clock speed has barely moved in twenty years.

THE REASON

Transistors kept coming. Speed did not.

Moore's law

Transistor count doubles every ~2 years.

Still roughly true.

Dennard scaling

Smaller transistors → less power each, so you could clock them faster for free.

Broke around 2005.

$$P \approx C \cdot V^2 \cdot f$$

Once V stopped falling, doubling f meant 4-8x the power in the same square centimetre.

It melts.

So the transistors were spent on more cores instead.

The programmer's job changed

● Before 2005

- Write serial code
- Wait 18 months
- It runs twice as fast
- Parallelism optional

● After 2005

- Write serial code
- Wait 18 months
- It runs at the same speed
- You use 1 core of 96

Processors got fast. Memory did not.

LEVEL	TYPICAL SIZE	LATENCY	FEELS LIKE
Registers on the core	~2 KB	0–1 cycles	1 second
L1 cache per core, split I/D	32–64 KB	~4 cycles	4 seconds
L2 cache per core	0.5–2 MB	~14 cycles	14 seconds
L3 cache shared by all cores	8–256 MB	~40 cycles	40 seconds
DRAM main memory	64 GB – 2 TB	~200 cycles	3 minutes
NVMe SSD local scratch	1–10 TB	~200 000 cycles	2.5 days
Network / parallel FS	100 TB +	~2 000 000 cycles	3 weeks

If one cycle were one second

A register read: 1 second. Main memory: 3 minutes. The shared filesystem: 3 weeks.

Most programs are waiting, not computing

A main-memory access costs ~200 cycles. That is ~750 instruction slots the processor could have used, and did not.

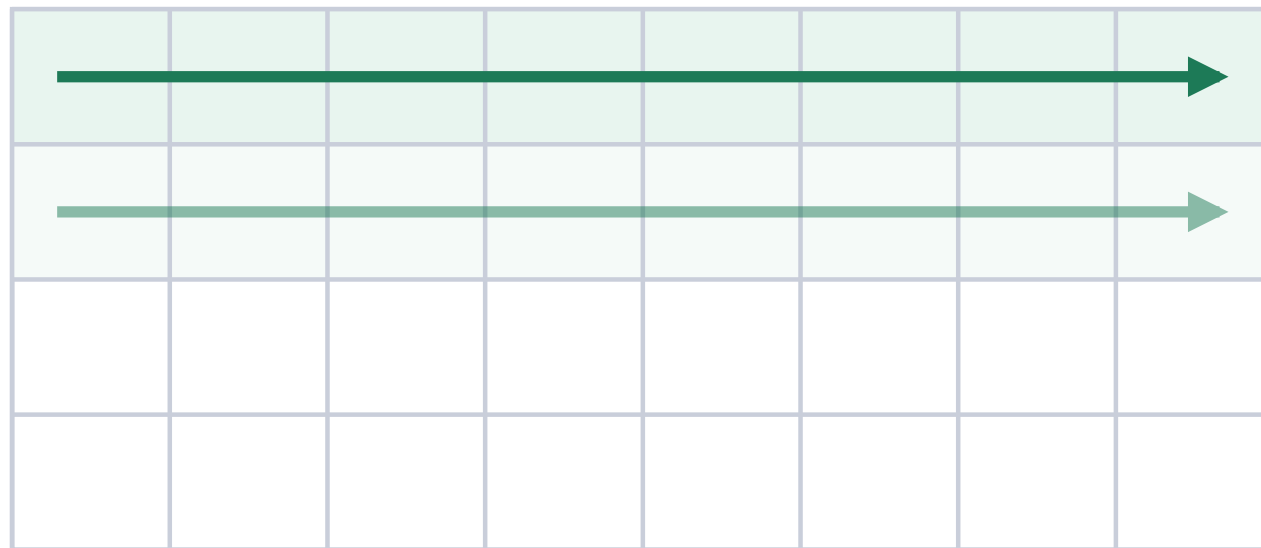
MEMORY WALL

Good HPC code is mostly the art of **waiting less**.

Loop order is worth 8x before you parallelise anything

Row order: j innermost

```
for i: for j: a[i][j]
```



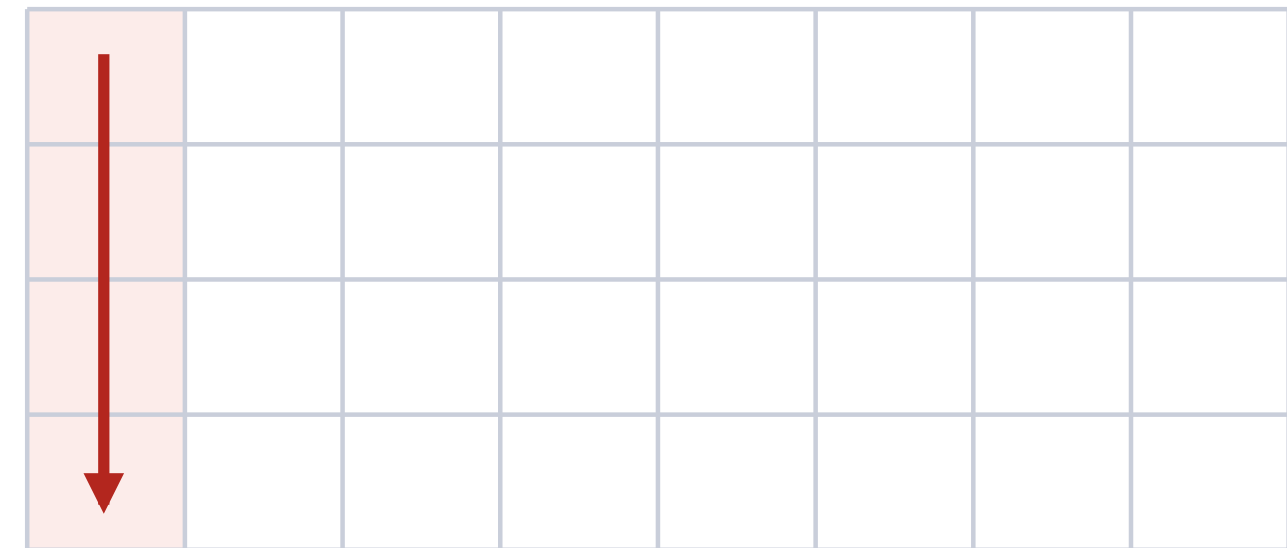
1 cache miss brings 8 useful doubles

Consecutive elements sit next to each other in memory. One 64-byte line fetch serves the next eight iterations, and the prefetcher sees the pattern and stays ahead of the loop.

~8 bytes of traffic per element

Column order: i innermost

```
for j: for i: a[i][j]
```



1 cache miss brings 1 useful double

Each step jumps a whole row. Every access pulls a fresh 64-byte line to use 8 bytes of it, and the line is usually evicted before the loop comes back for its neighbours.

~64 bytes of traffic per element: 8x worse

What actually stops you

- **Power wall**

Cannot raise clock speed without melting the chip.

- **Memory wall**

Processors outran memory. Data movement now dominates.

- **ILP wall**

The hardware already found most of the parallelism inside one instruction stream.

Parallelism is not an optimisation.

It is the only remaining source of performance.

Four units, one argument

Unit 1 How fast could it possibly go?

Unit 2 Use all the cores, then all the nodes.

Unit 3 Use the accelerators.

Unit 4 Use someone else's datacentre, and train a model on it.

Next: how to say in advance how much faster it can get

Speedup, efficiency, Amdahl, and the roofline.